Mass loss model for pebble aggregate samples in DiMES

Erick Martinez Loran

August 2026

1 The mass loss is proportional to the incident heat load

$$\frac{dm}{dt} = cq_{\text{incident}} = cq_{\parallel}A_w, \tag{1}$$

where $q_{\parallel}$ is the parallel heat load at the divertor and A_w is the wetted area: $A_w(t) = Dh(t)/2$, with $h(t)$ the height of the rod exposed to the plasma at time t.

Consider the incident heat loss as

$$q_{\parallel} = \gamma n_e k_{\text{B}} T_e c_s, \tag{2}$$

where γ is the transmission coefficient in the sheath, k_{B} is Boltzmann's constant, c_s is the ion sound speed, T_e, and n_e are the electron temperature and electron density at the sheath edge, respectively.

The ion saturation current density J_{sat} is the maximum current that can be drawn to a material surface (such as a Langmuir probe or vacuum chamber wall) and is defined as follows:

$$J_{\text{sat}} \equiv en_{\text{se}}c_s \tag{3}$$

Substitute $n_{\text{se}}c_s = J_{\text{sat}}/e$ into the sheath transmission equation Eq. (2):

$$q_{\parallel} = \frac{\gamma J_{\text{sat}} k_{\text{B}} T_e}{e}. \tag{4}$$

Multiply Eq. (4) by the wetted area A_w and plug it into Eq. (1):

$$\frac{dm}{dt} = cA_w \frac{\gamma J_{\text{sat}} k_{\text{B}} T_e}{e} \tag{5}$$

We propose that $A_w(t)J_{\text{sat}}(t)$ is the current measured at DiMES:

$$\frac{dm}{dt} = c\gamma k_{\text{B}} I(t) T_e(t).$$

The prefactor $c\gamma k_{\text{B}}$ can be lumped into a single constant c

$$\boxed{\frac{dm}{dt} = cI(t)T_e(t)} \tag{6}$$

To calibrate the model, we measure the total mass change $m_{\text{L}} \equiv \Delta m = m_f - m_i$ before and after exposure to the plasma heat load. Then,

$$m_{\text{L}} = c\int_0^t I(t)T_e(t)dt = cQ. \tag{7}$$

So:

$$c = \frac{m_{\text{L}}}{Q} \tag{8}$$

2 Estimating the uncertainty in c

The total uncertainty is given by:

$$\delta c = \left[\left(\frac{\delta m_{\mathrm{L}}}{m_{\mathrm{L}}} \right)^2 + \left(\frac{\delta Q}{Q} \right)^2 \right]^{1/2} \tag{9}$$

2.1 Q is estimated numerically

Q is estimated using either the trapezoid or Simpson's rule.

Simpson's rule

$$Q = \frac{\Delta t}{3} \sum_{i=0}^{N} w_i, I_i T_{e,i}, \qquad w_i \in \{1, 4, 2, 4, 2, \ldots, 4, 1\} \tag{10}$$

So each contribution is $q_i = \frac{\Delta t}{3} w_i I_i, T_{e,i}$:

$$Q = \sum_{i=0}^{N} q_i$$

The partial derivatives of q_i are

$$\frac{\partial q_i}{\partial I_i} = \frac{q_i}{I_i}, \quad \frac{\partial q_i}{\partial T_{e,i}} = \frac{q_i}{T_{e,i}}$$

So

$$\begin{aligned} \delta q_i &= \left[\left(\frac{q_i \delta I_i^2}{I_i} \right)^2 + \left(\frac{q_i \delta T_{e,i}^2}{T_{e,i}} \right)^2 \right]^{1/2} \\ &= q_i \left[\left(\frac{\delta I_i^2}{I_i} \right)^2 + \left(\frac{\delta T_{e,i}^2}{T_{e,i}} \right)^2 \right]^{1/2} \end{aligned}$$

The uncertainty in Q is

$$\delta Q = \left[\sum_i \left(\frac{\partial Q}{\partial q_i} \right)^2 \delta q_i^2 \right]^{1/2} = \left[\sum_i \delta q_i^2 \right]^{1/2}$$

Then,

$$\delta Q = \left\{ \sum_i q_i^2 \left[\left(\frac{\delta I_i}{I_i} \right)^2 + \left(\frac{\delta T_{e,i}}{T_{e,i}} \right)^2 \right] \right\}^{1/2}$$

Expanding q_i:

$$\delta Q = \left\{ \sum_i \left(\frac{\Delta t}{3} w_i I_i T_{e,i} \right)^2 \left(\frac{\delta I_i}{I_i} \right)^2 + \sum_i \left(\frac{\Delta t}{3} w_i I_i T_{e,i} \right)^2 \left(\frac{\delta T_{e,i}}{T_{e,i}} \right)^2 \right\}^{1/2}$$

Considering the systematic uncertainty on the current: $\delta I_i / I_i \equiv \epsilon_I$ (from considering a $\epsilon_I = 5\%$ tolerance resistor and $I = V/R \Rightarrow \delta I = I \delta R / R = \epsilon_I I$), and canceling terms:

$$\delta Q = \left\{ \sum_i \left(\frac{\Delta t}{3} w_i T_{e,i} I_i \epsilon_I \right)^2 + \sum_i \left(\frac{\Delta t}{3} w_i I_i \delta T_{e,i} \right)^2 \right\}^{1/2}$$

The first term is equal to $\epsilon_i Q$:

$$\boxed{\delta Q = \left\{ (\epsilon_I Q)^2 + \sum_i \left(\frac{\Delta t}{3} w_i I_i \delta T_{e,i} \right)^2 \right\}^{1/2}} \tag{11}$$

This can be implemented in Python as follows:

Code 1: Example implementation of δQ in Python

```python
import numpy as np
from scipy.integrate import simpson

# --- Inputs ---
# t, I, Te, dTe are 1D arrays of equal length N
# (N-1 must be even for pure Simpson)
# ddm = uncertainty in delta_m, dm = delta_m value
epsilon_I = 0.05  # delta_R / R

# --- Integral via Simpson's rule ---
integrand = I * Te
Q = simpson(integrand, x=t)

# --- Build Simpson weights explicitly ---
N = len(t)
dt = t[1] - t[0]  # assumes uniform spacing

def simpson_weights(N, dt):
    """Returns Simpson's 1/3 rule weights for N points (N must be odd)."""
    assert N % 2 == 1, "Need odd number of points (even number of intervals)"
    w = np.ones(N)
    w[1:-1:2] = 4   # odd indices
    w[2:-2:2] = 2   # even indices (interior)
    return w * dt / 3

w = simpson_weights(N, dt)

# --- Uncertainty in Q ---
dQ_sys  = epsilon_I * Q                       # systematic (R calibration)
dQ_stat = np.sqrt(np.sum((w * I * dTe)**2))  # statistical (Te errors)
dQ      = np.sqrt(dQ_sys**2 + dQ_stat**2)

# --- Uncertainty in c ---
c = dm / Q
rel_unc_c = np.sqrt((ddm / dm)**2 + (dQ / Q)**2)
delta_c   = c * rel_unc_c

print(f"Q        = {Q:.4e}")
print(f"dQ (sys) = {dQ_sys:.4e}  ({100*dQ_sys/Q:.1f}%)")
print(f"dQ (stat)= {dQ_stat:.4e}  ({100*dQ_stat/Q:.1f}%)")
print(f"dQ total = {dQ:.4e}  ({100*dQ/Q:.1f}%)")
print(f"c        = {c:.4e}")
print(f"delta_c  = {delta_c:.4e}  ({100*rel_unc_c:.1f}%)")
```

3 Expanding the uncertainty in the rate of mass loss

The rate of mass loss is given by Eq. (6):

$$r(t) = \frac{dm}{dt} = cI(t)T_e(t). \tag{12}$$

The constant c, the current $I(t)$, and the electron temperature $T_e(t)$ have associated uncertainties δc, $\delta I = \epsilon_I$, and δT_e.

Writing Eq. (12) explicitly

$$r(t) = \frac{m_{\rm L}}{\int_0^\tau I(t)T_e(t)dt} I(t)T_e(t).$$

Take the logarithm

$$\begin{aligned} \ln r(t) &= \ln(m_{\rm L}) + \ln[I(t)] + \ln[T_e(t)] - \ln\left(\int_0^\tau I(t)T_e(t)dt\right) \\ &= \ln m_{\rm L} + \ln I + \ln T_e - \ln Q \end{aligned} \tag{13}$$

Take the partial derivative

$$\frac{\partial \ln r}{\partial m_{\rm L}} = \frac{1}{r}\frac{\partial r}{\partial m_{\rm L}} = \frac{1}{m_{\rm L}} \tag{14}$$

Varying each error source, **one at a time**:

3.1 Variation 1: $\delta m_{\rm L}$ (independent)

$$\left(\frac{\delta r}{r}\right)_{m_{\rm L}} = \frac{1}{r}\left(\frac{\partial r}{\partial m_{\rm L}}\right)\delta m_{\rm L} = \frac{\delta m_{\rm L}}{m_{\rm L}}. \tag{8}$$

3.2 Variation 2: $\delta R/R$ (systematic, affects all I_i and $I(t)$

Every I_i has propagates the uncertainty as $I_i \to I_i(1+\epsilon)$ and $I(t) \to I(t)(1+\epsilon)$.

We are assuming that the error is systematic: R **has a fractional error** $\epsilon = \delta R/R$.

Since $I = V/R$, a resistor that is **larger than assumed** means the **current is smaller than assumed**:

$$I_{\rm true} = \frac{V}{R_{\rm true}} = \frac{V}{R(1+\epsilon)} \approx I(1-\epsilon)$$

So strictly speaking $\delta I/I = -\epsilon$. However, for **uncertainty propagation, we only care about the magnitude** of the effect, not the sign. Writing $I \to I(1+\epsilon)$, with $\epsilon = \pm\delta R/R$ is just a convenient way to track how a fractional shift propagates through the formula.

Then, $Q \to Q(1+\epsilon)$:

$$Q = \int_0^\tau I(t)T_e(t)dt \to \int_0^\tau I(t)(1+\epsilon)T_e(t)dt = (1+\epsilon)\int_0^\tau I(t)T_e(t)dt = Q(1+\epsilon).$$

This causes cancelation in r: Substitute both shifts in r:

$$r = cI(t)T_e(t) = \frac{m_{\rm L}}{Q}I(t)T_e(t)$$

$$r \to \frac{m_{\rm L}}{Q(1+\epsilon)}I(t)(1+\epsilon)T_e(t) = \frac{m_{\rm L}}{Q}I(t)T_e(t)\frac{(1+\epsilon)}{(1+\epsilon)} = r$$

The $(1+\epsilon)$ factors cancel **exactly**, so r is completely insensitive to ϵ

3.3 Variation 3: δT_e

Assuming Simpson's rule, $Q = \frac{\Delta t}{3} \sum_i w_i I_i T_{e,i}$. Then,

$$\ln r_k = \ln m_{\rm L} + \ln I_k + lnT_{e,k} - \ln\left(\frac{\Delta t}{3}\sum_j w_j I_j T_{e,j}\right) \tag{15}$$

T_e appears **twice**:

- Once **explicitly** in $+\ln T_{e,l}$
- Once **implicitly** inside $-\ln Q$, at every time step j, including $j = k$

Case $j \neq k$: $T_{e,j}$ appears only inside $\ln Q$

$$\frac{\partial \ln r_k}{\partial T_{e,j}} = -\frac{\partial \ln Q}{\partial T_{e,j}} = -\frac{1}{Q}\frac{\partial Q}{\partial T_{e,j}} = -\frac{w_j I_j \frac{\Delta t}{3}}{Q} \tag{9}$$

Case $j = k$: $T_{e,j}$ appears in both terms $\ln T_{e,k}$, and $-\ln Q$ simultaneously

$$\frac{\partial \ln r_k}{\partial T_{e,k}} = \underbrace{\frac{\partial \ln T_{e,k}}{\partial T_{e,k}}}_{\text{explicit term}} - \underbrace{\frac{\partial \ln Q}{\partial T_{e,k}}}_{\text{implicit term}}$$

$$= \frac{1}{T_{e,k}} - \frac{w_j I_j \frac{\Delta t}{3}}{Q} \tag{16}$$

Multiplying through by $\delta T_{e,k}$ to express as a relative uncertainty:

$$\frac{\partial \ln r_k}{\partial T_{e,k}}\delta T_{e,k} = \left[1 - \frac{w_k I_k T_{e,k}\frac{\Delta t}{3}}{Q}\right]\frac{\delta T_{e,k}}{T_{e,k}} \tag{17}$$

Define

$$\boxed{\alpha_k \equiv \frac{w_k I_k T_{e,k}\frac{\Delta t}{3}}{Q}} \tag{18}$$

3.4 Full T_e uncertainty

We can unify both cases cleanly

$$\left(\frac{\delta r_k}{r_k}\right)^2_{T_e} = \sum_j \left(\frac{\partial \ln r_k}{\partial T_{e,j}}\delta T_{e,j}\right)^2 = \sum_j \left(\frac{\partial}{\partial T_{e,j}}\left[\ln T_{e,k} - \ln Q\right]\delta T_{e,j}\right)^2 \tag{19}$$

where

$$\frac{\partial \ln r_k}{\partial T_{e,j}} = \begin{cases} \dfrac{1-\alpha_k}{T_{e,k}}, & j = k \\ -\dfrac{w_j I_j \frac{\Delta t}{3}}{Q}, & j \neq k \end{cases} \tag{20}$$

So the sum over **all** j naturally splits into the two cases:

$$\boxed{\left(\frac{\delta r_k}{r_k}\right)^2 = (1-\alpha_k)^2\left(\frac{\delta T_{e,k}}{T_{e,k}}\right)^2 + \sum_{j\neq k}\left(\frac{w_j I_j \frac{\Delta t}{3}}{Q}\right)^2 \delta T_{e,j}} \tag{21}$$

This can be easily implemented in **Python**

Code 2: Sample implementation of δr_k in Python

```python
import numpy as np

# --- Trapezoid weights ---
def trapezoid_weights(t):
    """Non-uniform spacing supported."""
    dt = np.diff(t)
    w = np.zeros(len(t))
    w[:-1] += dt / 2
    w[1:]  += dt / 2
    return w

w = trapezoid_weights(t)

# --- Integral ---
Q = np.sum(w * I * Te)

# --- Alpha: fractional contribution of each time step ---
alpha = w * I * Te / Q  # shape (N,), sums to 1

# --- Uncertainty in r(t) ---
r = c * I * Te

# Term 1: delta_m (scalar)
frac_dm = (ddm / dm)**2

# Terms 2+3: T_e contributions, split at each t
# For each index k, the local term is (1 - alpha[k])^2 * (dTe[k]/Te[k])^2
# and the sum over j != k of (w[j]*I[j]*dTe[j]/Q)^2

full_Te_sum = np.sum((w * I * dTe / Q)**2)  # sum over ALL j
local_Te    = (w * I * dTe / Q)**2          # contribution of each j
                                            # to sum

# For index k:
#   sum_{j != k} = full_Te_sum - local_Te[k]
#   local term   = (1 - alpha[k])^2 * (dTe[k]/Te[k])^2

frac_Te_nonlocal = full_Te_sum - local_Te         # array, sum over j!=t
frac_Te_local    = (1 - alpha)**2 * (dTe / Te)**2 # array, local t term

delta_r_over_r = np.sqrt(frac_dm + frac_Te_local + frac_Te_nonlocal)
delta_r = r * delta_r_over_r
```

4 Propagating uncertainty in $m_{\mathrm{lost}}(t)$

The cumulative eroded mass up to time step k is:

$$m_k = cS_k = m_{\mathrm{L}} \left(\frac{S_k}{S} \right), \tag{22}$$

where $S_k \equiv \sum_{i=0}^{k} w_i, I_i, T_{e,i}$, and $S \equiv \sum_{i=0}^{N} w_i, I_i, T_{e,i} = Q/(\Delta t/3)$. For simplicity, we designate m as the emitted (or lost) mass m_{lost}.

Taking the log:

$$\ln m_k = \ln m_{\mathrm{L}} + \ln S_k - \ln S \tag{23}$$

4.1 Variations

4.1.1 δm_{L} (independent)

$$\frac{\partial \ln m_k}{\partial m_{\mathrm{L}}} = \frac{1}{m_{\mathrm{L}}} \Longrightarrow \left(\frac{\delta m_k}{m_k}\right)_{m_{\mathrm{L}}} = \frac{\delta m_{\mathrm{L}}}{m_{\mathrm{L}}} \tag{24}$$

4.1.2 $\delta R/R$ (systematic, all I_i scale together

Every $I_i \to I_i(1+\epsilon)$, so $S_k \to S_k(1+\epsilon)$, and $S \to S(1+\epsilon)$.

$$\begin{aligned}\delta \ln m_k &= \ln\left[S_k(1+\varepsilon)\right] - \ln\left[S(1+\varepsilon)\right] - \ln S_k + \ln S \\ &= \ln(1+\epsilon) - \ln(1+\epsilon) = 0\end{aligned}$$

Cancels exactly, same argument as for r_k.

4.1.3 $\delta T_{e,j}$ – Three cases now

Since $\ln m_k = \ln m_{\mathrm{L}} + \ln S_k - \ln S$, varying $T_{e,j}$, gives

$$\frac{\partial \ln m_k}{\partial T_{e,j}} = \frac{\partial \ln S_k}{\partial T_{e,j}} - \frac{\partial \ln S}{\partial T_{e,j}} \tag{25}$$

The key difference from r_k: there is **no explicit** $\ln T_{e,k}$ **term**. T_e only appears inside the two S_k and S. *Why do we need to vary over all values of j?*: Because m_k depends on **all** $T_{e,j}$ values, not just the local one at $j = k$: Look at $m_k = m_{\mathrm{L}}(S_k/S) = m_{\mathrm{L}}\left(\sum_{i=0}^{k} w_i I_i T_{e,j}\right) / \left(\sum_{i=0}^{N} w_i I_i T_{e,j}\right)$. Every single $T_{e,j}$ for $j = 0, \ldots, N$ appears somewhere in this expression.

Case 1: $j > k$ – appears only in S, not in S_k

$$\begin{aligned}\frac{\partial \ln m_k}{\partial T_{e,j}} &= \frac{1}{S_k}\frac{\partial S_k}{\partial T_{e,j}} - \frac{1}{S}\frac{\partial S}{\partial T_{e,j}} \\ &= \frac{1}{S_k}\sum_{i=0}^{k} \underbrace{\frac{\partial}{\partial T_{e,j}}\left(w_i I_i T_{e,i}\right)}_{=0 \text{ since } j>k} - \frac{1}{S}\sum_{i=0}^{N} \underbrace{\frac{\partial}{\partial T_{e,j}}\left(w_i I_i T_{e,i}\right)}_{=\delta_{ij}} \\ &= -\frac{w_j I_j}{S} \\ \Rightarrow \frac{\partial \ln m_k}{\partial T_{e,j}}\delta T_{e,j} &= -\frac{w_j I_j}{S}\delta T_{e,j}\end{aligned}$$

Case 2: $j < k$ – appears in both S_k, and S

$$\begin{aligned}\frac{\partial \ln m_k}{\partial T_{e,j}} &= \frac{1}{S_k}\sum_{i=0}^{k} \underbrace{\frac{\partial}{\partial T_{e,j}}\left(w_i I_i T_{e,i}\right)}_{=\delta_{ij}} - \frac{1}{S}\sum_{i=0}^{N} \underbrace{\frac{\partial}{\partial T_{e,j}}\left(w_i I_i T_{e,i}\right)}_{=\delta_{ij}} \\ &= \frac{w_j I_j}{S_k} - \frac{w_j I_j}{S} = w_j I_j\left(\frac{1}{S_k} - \frac{1}{S}\right) \\ \Rightarrow \frac{\partial \ln m_k}{\partial T_{e,j}}\delta T_{e,j} &= w_j I_j\left(\frac{1}{S_k} - \frac{1}{S}\right)\delta T_{e,j}\end{aligned}$$

Case 3: $j = k$ – also appears in both S_k, and S

$$\frac{\partial \ln m_k}{\partial T_{e,k}} = \frac{1}{S_k}\sum_{i=0}^{k}\underbrace{\frac{\partial}{\partial T_{e,k}}\left(w_i I_i T_{e,i}\right)}_{=\delta_{ik}} - \frac{1}{S}\sum_{i=0}^{N}\underbrace{\frac{\partial}{\partial T_{e,k}}\left(w_i I_i T_{e,i}\right)}_{=\delta_{ik}}$$

$$= \frac{w_k I_k}{S_k} - \frac{w_k I_k}{S} = w_k I_k\left(\frac{1}{S_k} - \frac{1}{S}\right)$$

$$\Rightarrow \frac{\partial \ln m_k}{\partial T_{e,k}}\delta T_{e,k} = w_k I_k\left(\frac{1}{S_k} - \frac{1}{S}\right)\delta T_{e,k}$$

The full expression for $\frac{\partial \ln m_k}{\partial T_{e,j}}\delta T_{e,j}$ is

$$\frac{\partial \ln m_k}{\partial T_{e,k}}\delta T_{e,j} = \begin{cases} w_j I_j\left(\frac{1}{S_k} - \frac{1}{S}\right)\delta T_{e,j} & j \le k \\ -\frac{w_j I_j}{S}\delta T_{e,j} & j > k \end{cases} \tag{26}$$

4.1.4 Full result

$$\boxed{\frac{\delta m_k}{m_k} = \left\{\left(\frac{\delta m_\mathrm{L}}{m_\mathrm{L}}\right)^2 + \sum_{j\le k}\left[w_j I_j\left(\frac{1}{S_k} - \frac{1}{S}\right)\delta T_{e,j}\right]^2 + \sum_{j>k}\left[\frac{w_j I_j}{S}\delta T_{e,j}\right]^2\right\}^{1/2}} \tag{27}$$

This can be easily implemented in **Python**:

Code 3: Example code to estimate δm_k in Python

```python
# S_k: cumulative sum up to each index k
S_k = np.cumsum(w * I * Te)   # shape (N,)
S   = S_k[-1]                 # full sum = Q

# Precompute per-step weight * current
wI = w * I                    # shape (N,)

# For each k, build delta_m_k/m_k
frac_dm = (ddm / dm)**2       # scalar

frac_Te = np.zeros(N)
for k in range(N):
    past   = np.arange(0, k+1)          # j <= k
    future = np.arange(k+1, N)          # j > k
    term_past   = np.sum((wI[past]   * (1/S_k[k] - 1/S) * dTe[past])**2)
    term_future = np.sum((wI[future] * dTe[future] / S)**2)
    frac_Te[k]  = term_past + term_future

delta_m_over_m = np.sqrt(frac_dm + frac_Te)
```

5 Change of A_w with incident heat load

We assume that the wetted area A_w decreases with exposure to heat load. This should be proportional to the mass loss:

$$A_\mathrm{w} \equiv Dh(t)$$

where D is the diameter of the rod and $h(t)$ is the height of the rod exposed to the plasma at time t_k. $h(t)$ decreases with mass loss:

$$\Delta h(t) \propto m_{\text{lost}}(t)$$

with m_{lost} as defined in Eq. (22). We assume D remains constant.

The volume of the rod lost during exposure is $V(t) = \pi D^2 h(t)/4$. Assuming a constant effective density ρ, $m_{\text{lost}}(t)/\rho = \pi D^2 \Delta h(t)/4$. Then,

$$\Delta h(t) = \frac{4}{\rho \pi D^2} m_{\text{lost}}(t)$$

And the wetted area is given by

$$A_w(t) = D\left(\frac{4}{\rho_{\text{rod}} \pi D^2}\right) m_{\text{lost}}(t) = \frac{4}{\rho_{\text{rod}} \pi D} m_{\text{lost}}(t) \tag{28}$$

5.1 Uncertainty propagation in A_w

The quantities ρ_{rod}, D, and m_{lost} have associated uncertainties $\delta\rho_{\text{rod}}$, δD, and δm_{lost}. So the uncertainty in A_w is given by

$$\delta A_w(t) = A_w(t) \left\{ \left(\frac{\delta D}{D}\right)^2 + \left(\frac{\delta \rho_{\text{rod}}}{\rho_{\text{rod}}}\right)^2 + \left(\frac{\delta m_{\text{lost}}}{m_{\text{lost}}}\right)^2 \right\}^{1/2} \tag{29}$$

5.2 Surface recession rate

The surface recession rate for the wetted surface

$$v \equiv \frac{1}{\rho_{\text{rod}} A_w(t)} \frac{dm(t)}{dt} \tag{30}$$

Plugging in Eq. (28):

$$v(t) = \frac{1}{\rho_{\text{rod}}} \left(\frac{\rho_{\text{rod}} \pi D}{4 m_{\text{lost}}(t)}\right) r(t) = \frac{\pi D}{4} \frac{r(t)}{m_{\text{lost}}(t)}$$

Which can be written **explicitly** in terms of $I(t)$, and $T_e(t)$ as

$$v(t) = \frac{\pi D}{4} \frac{1}{c \int_0^t I(t') T_e(t') dt} c I(t) T_e(t) \tag{31}$$

Expressing it in numeric form:

$$v_k = \frac{\pi D}{4} \frac{I_k T_{e,k}}{Q_k} \tag{32}$$

5.3 Uncertainty in surface recession rate

Start with the expression in Eq. (32):

$$v_k = \frac{\pi D}{4} \frac{I_k T_{e,k}}{Q_k}.$$

Define

$$X_i \equiv I_i T_{e,i}, \quad Q_k \equiv \sum_{i=0}^{k} w_i X_i, \tag{33}$$

where we have lumped $\Delta t/3$ in the quadrature weights w_i.

The propagation of the uncertainty of $f(u_1, \ldots, u_n)$ is given by

$$\sigma_f^2 = \sum_m \left(\frac{\partial f}{\partial u_m} \right)^2 \sigma_m^2$$

Assume $\delta V = 0$, so that the error in $I = V/R$ is systematic from $\delta R/R = \epsilon$. Because R is constant across measurements, **even with a nonzero ϵ, it does not contribute to δ_k at all.**

Here is why, if $Ij = V_j/R_j$ with the *same* R is used for every sample, then substitute it directly into ν_k before ever differentiating:

$$\nu_k = \frac{\pi D}{4} \frac{I_k T_{e,k}}{Q_k} = \frac{\pi D}{4} \frac{(V_k/R) T_{e,k}}{\sum_i^k w_i (V_i/R) T_{e,i}} = \frac{\pi D}{4} \frac{V_k T_e, k}{\sum_i^k w_i V_i T_{e,i}}$$

R factors out of every term in the sum S_k and out of the numerator identically, so it cancels completely –algebraically, before taking any derivative.

5.3.1 The D partial

$\nu_k \propto D$ exactly, so

$$\frac{\partial \nu_k}{\partial D} = \frac{\nu_k}{D} \tag{34}$$

trivial.

5.3.2 The X_j partial

Rather than differentiating directly with respect to $T_{e,j}$, it is cleaner to first get $\partial \nu_k / \partial X_j$, then use the chain rule through $X_j = I_j T_{e,j}$.

Write $\nu_k = \frac{\pi D}{2} X_k Q_k^{-1}$ and use the product rule

$$\frac{\partial \nu_k}{\partial X_j} = \frac{\pi D}{4} \left[\frac{\partial X_k}{\partial X_j} \cdot \frac{1}{Q_k} + X_k \cdot \frac{\partial \left(Q_k\right)^{-1}}{\partial X_j} \right]$$

Two pieces to work out:

- $\frac{\partial X_k}{\partial X_j} = \delta_{jk}$ (the Kronecker delta – X_k only depends on itself, so this is 1 if $j = k$, 0 otherwise). **This is the term that captures the numerator/denominator correlation** – X_k appears explicitly in the numerator , so differentiating the numerator with respect to X_j only "sees" it when $j = k$.
- $\frac{\partial Q_k}{\partial X_j} = w_j$ for $j \leq k$ (since $Q_k = \sum_{i=0}^{k} w_i X_i$ is linear in each X_i), so by the chain rule, $\frac{\partial \left(S_k\right)^{-1}}{\partial X_j} = -Q_k^{-2} \frac{\partial Q_k}{\partial X_j} = -\frac{w_j}{Q_k^2}$

Putting these together:

$$\frac{\partial \nu_k}{\partial X_j} = \frac{\pi D}{4} \left[\frac{\delta_{jk}}{Q_k} - \frac{w_j X_k}{Q_k^2} \right], \quad j \leq k \tag{35}$$

5.3.3 Chain rule down to $T_{e,j}$

Since $X_j = I_j T_{e,j}$, with no error through I_j (systematic error $\delta R/R$ cancels out and $\delta V_j = 0$ assumed),

$$\frac{\partial v_k}{\partial T_{e,j}} = \frac{\partial v_k}{\partial X_j} \cdot \frac{\partial X_j}{\partial T_{e,j}} = \frac{\partial v_k}{\partial X_j} \cdot I_j$$

so in variance form

$$\frac{\partial v_k}{\partial T_{e,j}} \delta T_{e,j} = \frac{\pi D}{4} \left[\frac{\delta_{jk}}{Q_k} - \frac{w_j X_k}{Q_k^2} \right] I_j \delta T_{e,j}, \quad j \le k$$

It is cleanest to just define $\delta X_j \equiv I_j \delta T_{e,j}$ and treat $\left(\frac{\partial v_k}{\partial X_j}\right)^2 \delta X_j^2$ as the contribution.

5.3.4 Assemble the total variance

Sum the independent contributions from D and each $T_{e,j}$, $j = 0, 1, \ldots, k$:

$$\boxed{\delta v_k = \left\{ \left(\frac{v_k}{D}\right)^2 \delta D^2 + \sum_{j=0}^{k} \left[\frac{\pi D}{4} \left(\frac{\delta_{jk}}{Q_k} - \frac{w_j X_k}{Q_k^2} \right) \right]^2 \delta X_j^2 \right\}^{1/2}} \tag{36}$$

This can be implemented in Python as

Code 4: Example implementation of the recession rate calculation in Python

```
"""
Uncertainty propagation for the DiMES surface recession rate.

    nu_k = (pi * D / 2) * X_k / Q_k,   X_i = I_i * Te_i
    Q_k  = numerical integral of X over [t_0, t_k], via 'trapezoid' or
           'simpson' composite quadrature -- NON-UNIFORM time grid t.
    I_i treated as exact (dV = 0); dR/R contributes nothing (see prior
    derivation) and is not modeled.

Quadrature coefficients dQ_k/dX_j  (t is now an array, not a scalar dt)
-------------------------------------------------------------------------
trapezoid:  dQ_k/dX_0 = (t_1-t_0)/2
            dQ_k/dX_j = (t_{j+1}-t_{j-1})/2   (0<j<k)
            dQ_k/dX_k = (t_k-t_{k-1})/2

simpson:    composite Simpson over PANELS of 3 consecutive points, using
            the unequal-interval formula (reduces to the standard
            h/3, 4h/3, h/3 weights when the panel's two sub-intervals are
            equal). For odd k, a plain trapezoidal correction (using the
            true local width t_k - t_{k-1}) is added for the leftover
            interval, exactly as in the uniform-grid version.
"""

import numpy as np


def quad_coeffs(k, t, method="trapezoid"):
    """dQ_k/dX_j for j = 0..k, for the chosen quadrature method.
    t : array_like of sample times (length >= k+1), need not be uniform."""
    t = np.asarray(t, dtype=float)
```

```
    c = np.zeros(k + 1)
    if k == 0:
        return c

    if method == "trapezoid":
        c[0] = (t[1] - t[0]) / 2.0
        c[1:k] = (t[2:k + 1] - t[0:k - 1]) / 2.0
        c[k] = (t[k] - t[k - 1]) / 2.0
        return c

    if method == "simpson":
        m = k if k % 2 == 0 else k - 1  # largest even index <= k
        for start in range(0, m, 2):
            h0 = t[start + 1] - t[start]
            h1 = t[start + 2] - t[start + 1]
            w0 = (h0 + h1) / 6.0 * (2.0 - h1 / h0)
            w1 = (h0 + h1) / 6.0 * (h0 + h1) ** 2 / (h0 * h1)
            w2 = (h0 + h1) / 6.0 * (2.0 - h0 / h1)
            c[start] += w0
            c[start + 1] += w1
            c[start + 2] += w2
        if k % 2 == 1:  # trailing trapezoid correction, true local width
            half = (t[k] - t[k - 1]) / 2.0
            c[k - 1] += half
            c[k] += half
        return c

    raise ValueError(f"unknown method: {method}")


def cumulative_integral(X, t, method="trapezoid"):
    """Q_k for k = 0..N-1 (Q_0 = 0)."""
    N = len(X)
    Q = np.zeros(N)
    for k in range(1, N):
        c = quad_coeffs(k, t, method)
        Q[k] = np.dot(c, X[: k + 1])
    return Q


def propagate_analytic(D, dD, I, Te, dTe, t, method="trapezoid"):
    """
    Parameters
    ----------
    D, dD : float
    I : array_like, shape (N,)      -- treated as exact (dV = 0)
    Te, dTe : array_like, shape (N,)
    t : array_like, shape (N,)      -- sample times, non-uniform OK
    method : 'trapezoid' or 'simpson'

    Returns
    -------
    nu, dnu : ndarray, shape (N,)   (index 0 is undefined -- no interval yet)
    """
    I, Te, dTe, t = (np.asarray(a, dtype=float) for a in (I, Te, dTe, t))
    N = len(I)
```

```
    X = I * Te
    dX = I * dTe
    Q = cumulative_integral(X, t, method)

    nu = np.full(N, np.nan)
    dnu = np.full(N, np.nan)
    for k in range(1, N):
        Qk = Q[k]
        nu[k] = (np.pi * D / 4.0) * X[k] / Qk

        c = quad_coeffs(k, t, method)          # dQ_k/dX_j, j=0..k
        j = np.arange(k + 1)
        kronecker = (j == k).astype(float)
        dnu_dXj = (np.pi * D / 4.0) * (kronecker / Qk - c * X[k] / Qk ** 2)
        var_X = np.sum((dnu_dXj * dX[: k + 1]) ** 2)
        var_D = (nu[k] / D * dD) ** 2
        dnu[k] = np.sqrt(var_X + var_D)

    return nu, dnu


def propagate_montecarlo(D, dD, I, Te, dTe, t, method="trapezoid",
                         n_draws=20000, seed=0):
    rng = np.random.default_rng(seed)
    I, Te, dTe, t = (np.asarray(a, dtype=float) for a in (I, Te, dTe, t))
    N = len(I)

    D_s = rng.normal(D, dD, size=n_draws)
    Te_s = rng.normal(Te, dTe, size=(n_draws, N))
    X_s = I[None, :] * Te_s

    Q_s = np.zeros((n_draws, N))
    for k in range(1, N):
        c = quad_coeffs(k, t, method)
        Q_s[:, k] = X_s[:, : k + 1] @ c

    nu_s = np.full((n_draws, N), np.nan)
    nu_s[:, 1:] = (np.pi * D_s[:, None] / 2.0) * X_s[:, 1:] / Q_s[:, 1:]

    return np.nanmean(nu_s, axis=0), np.nanstd(nu_s, axis=0)


if __name__ == "__main__":
    N = 51
    rng = np.random.default_rng(1)
    # non-uniform grid: irregular but increasing spacings
    steps = 0.5 + rng.random(N - 1)
    t = np.concatenate([[0.0], np.cumsum(steps)])
    t /= t[-1]  # normalize to [0,1]

    I = 5.0 + 3.0 * np.sin(np.pi * t)
    Te = 20.0 + 10.0 * t
    dTe = 0.10 * Te
    D, dD = 0.6, 0.02

    for method in ("trapezoid", "simpson"):
        print(f"\n=== {method} (non-uniform grid) ===")
```

```
145
146         X = I * Te
147         k_test = 21  # odd, exercises the boundary-correction branch
148         eps = 1e-6
149         Q0 = cumulative_integral(X, t, method)
150         analytic_c = quad_coeffs(k_test, t, method)
151         fd_c = np.zeros(k_test + 1)
152         for j in range(k_test + 1):
153             Xp = X.copy(); Xp[j] += eps
154             Qp = cumulative_integral(Xp, t, method)
155             fd_c[j] = (Qp[k_test] - Q0[k_test]) / eps
156         print("max |analytic dQ_k/dX_j - finite-diff| =",
157               np.max(np.abs(analytic_c - fd_c)))
158
159         nu, dnu = propagate_analytic(D, dD, I, Te, dTe, t, method)
160         nu_mc, dnu_mc = propagate_montecarlo(D, dD, I, Te, dTe, t, method)
161
162         print(f"{'k':>3} {'nu':>10} {'dnu (analytic)':>16} {'dnu (MC)':>10}")
163         for k in (1, N // 4, N // 2, 3 * N // 4, N - 1):
164             print(f"{k:3d} {nu[k]:10.4f} {dnu[k]:16.4f} {dnu_mc[k]:10.4f}")
```